MACHINE LEARNING

>:_ FACULTAD DE INGENIERÍA

Universidad Rafael Landívar
Identidad Jesuita en Guatemala

POSGRADOS LANDÍVAR

Laboratorio 2.4

KNN

1. Tratamiento del dataset

Cargue el módulo de KNN de sklearn y cargue el dataset breast_cancer.csv como un dataframe de pandas. Analice el desbalance de las clases y además escale sus variables

In [1]:

```
from sklearn.neighbors import KNeighborsClassifier
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

dataset=pd.read_csv('breast_cancer.csv')

dataset
```

Out[1]:

	mean.radius	mean.texture	mean.perimeter	mean.area	mean.smoothness	mean.compactness	mean.concavity	mean.co
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	
...	...	...	...	...	...	...	...	
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	
566	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	
567	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	

569 rows × 31 columns

In [2]:

```
dataset["cancer"].value_counts()
```

Out[2]:

```
1    357
0    212
Name: cancer, dtype: int64
```

In [3]:

```
num_y=dataset["cancer"].value_counts()

min_num=min(num_y)

min_class=num_y[num_y==min(num_y)].index[0]

undersamp=dataset[dataset["cancer"]!=min_class].sample(min_num,random_state=10)
```

```
dataset=pd.concat([undersamp,dataset[dataset["cancer"]==min_class]],axis=0)

dataset["cancer"].value_counts()
```

Out[3]:

```
1    212
0    212
Name: cancer, dtype: int64
```

In [4]:

```
x_data=dataset.drop(['cancer'], axis=1)
y_data=dataset["cancer"]

scaler = MinMaxScaler()
scaler.fit(x_data)

x_data = scaler.transform(x_data)
```

2. Separación del dataset

Divida la data en training y test con el 80% de la data en train y agregue un random_state para que el ejercicio pueda ser replicable

In [6]:

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.metrics import confusion_matrix
import seaborn as sns


x_train,x_test,y_train,y_test=train_test_split(x_data,y_data,train_size=0.8,random_state
=14)
```

3. Entrenamiento del modelo

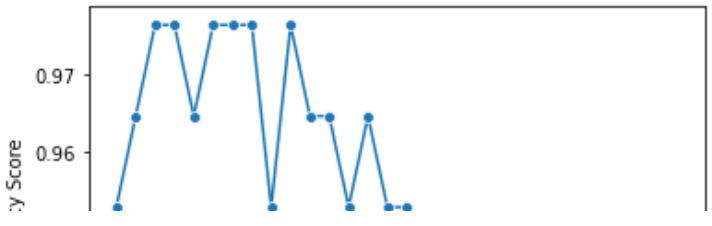
Entrene un modelo de KNN buscando el número óptimo de vecinos.

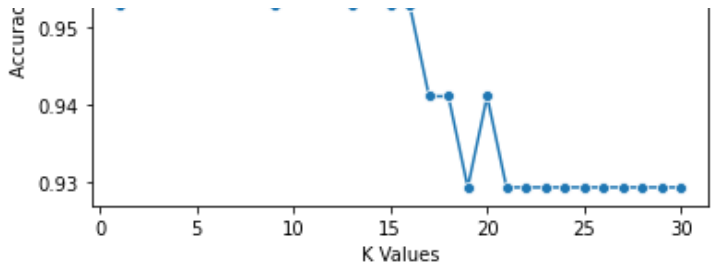
In [18]:

```
from sklearn.metrics import accuracy_score

k_values = [i for i in range (1,31)]
scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k,metric="manhattan")
    knn.fit(x_train, y_train)
    score = accuracy_score(y_test, knn.predict(x_test),normalize=True)
    scores.append(np.mean(score))

sns.lineplot(x = k_values, y = scores, marker = 'o')
plt.xlabel("K Values")
plt.ylabel("Accuracy Score")
```

Out[18]:

```
Text(0, 0.5, 'Accuracy Score')
```





In [28]:

```
classifier = KNeighborsClassifier(n_neighbors=5)
classifier.fit(x_train, y_train)

print(accuracy_score(y_train, classifier.predict(x_train),normalize=True))
print(accuracy_score(y_test, classifier.predict(x_test),normalize=True))
```

```
0.9646017699115044
0.9764705882352941
```

4. Evaluación de resultados

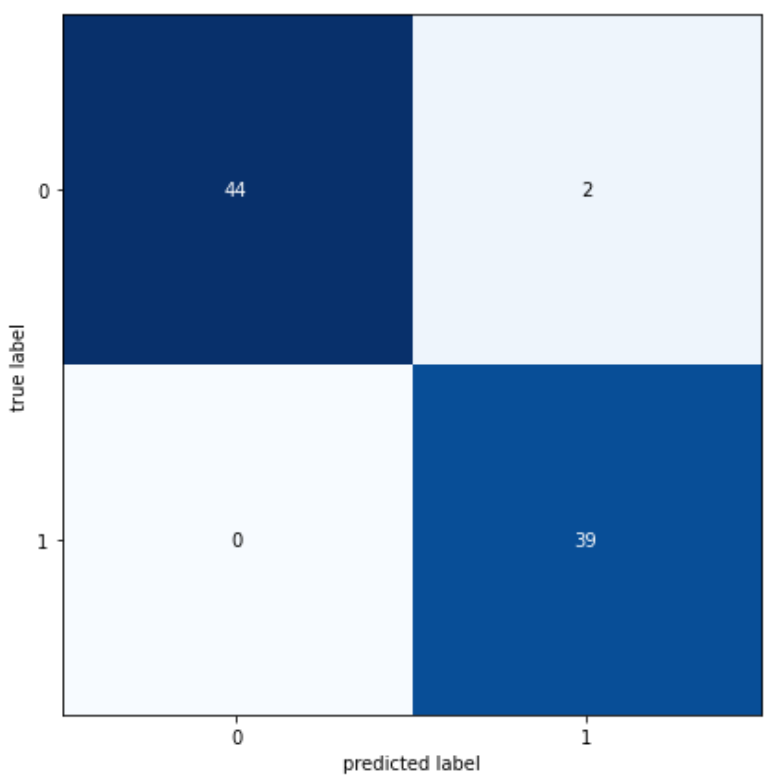
Obtenga la matriz de confusión y demás métricas de evaluación de su modelo clasificatorio. Además, grafique la curva ROC y encuentre el área debajo de la curva ROC para su test set.

In [29]:

```
from mlxtend.plotting import plot_confusion_matrix
y_pred = classifier.predict(x_test)

from sklearn.metrics import confusion_matrix, accuracy_score,classification_report
cm = confusion_matrix(y_test, y_pred)
plot_confusion_matrix(conf_mat=cm, figsize=(6,6), show_normed=False,)
plt.tight_layout()

accuracy_score(y_test, y_pred)
```

Out[29]:

```
0.9764705882352941
```

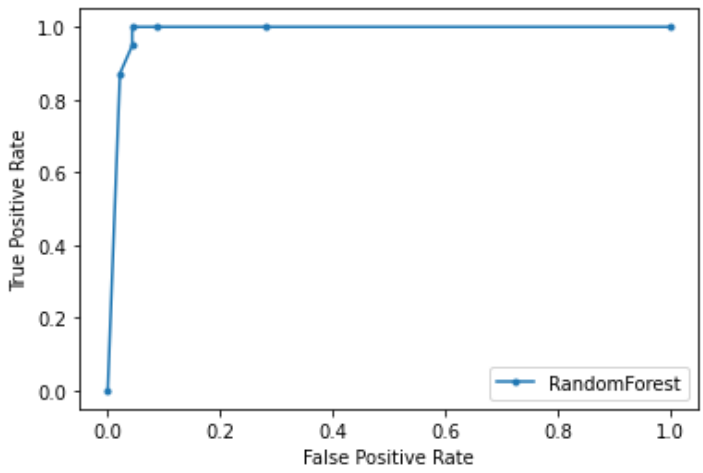


In [30]:

```
print(classification_report(y_test,round(pd.Series(classifier.predict(x_test)))))
```

```
              precision    recall  f1-score   support

           0       1.00      0.96      0.98        46
           1       0.95      1.00      0.97        39

    accuracy                           0.98        85
   macro avg       0.98      0.98      0.98        85
weighted avg       0.98      0.98      0.98        85
```

In [31]:

```
from sklearn.metrics import roc_auc_score,roc_curve
from matplotlib import pyplot

auc = roc_auc_score(y_test, classifier.predict_proba(x_test)[:,1])
print('AUC: %.3f' % auc)

fpr, tpr, _ = roc_curve(y_test,classifier.predict_proba(x_test)[:,1])
# plot the roc curve for the model
pyplot.plot(fpr, tpr, marker='.', label='RandomForest')
# axis labels
pyplot.xlabel('False Positive Rate')
pyplot.ylabel('True Positive Rate')
# show the legend
pyplot.legend()
# show the plot
pyplot.show()
```

```
AUC: 0.986
```